



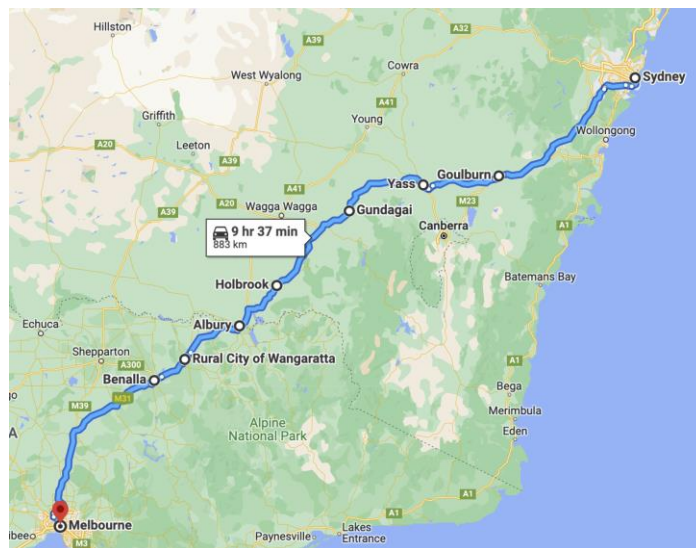
## Assignment 2

**Deadline: 6pm Thursday 8 October 2026**

### Problem: Electric Vehicle Charging

#### 1. General description

Electric vehicles (EVs) have gained increasing popularity in recent years, primarily due to their numerous advantages, including reduced carbon dioxide emissions, cost-effective operation, and enhanced driving experiences. In Australia, the adoption of EVs has been steadily on the rise, with an annual growth rate of approximately 10%. In the year 2022, 33,410 EVs sold in the Australian market<sup>1</sup>. However, there are only around 3700 public chargers, at just over 2,100 locations, across Australia<sup>2</sup>. Most purely electric cars have a battery capacity that allows them to travel between 300 and 600 kilometers on a single full charge. Imagine that you are driving an electric vehicle from Sydney to Melbourne or a city in between. Along the way, you'll have to recharge your car one or two times. Additional to charging time, you would have to wait for a long time for an available charger. Implementing a scheduling system capable of managing EV charging demands along the route would greatly enhance efficiency, reducing both charging and waiting times.



This assignment is to develop a system that can simulate EV charging allocation and balancing.

<sup>1</sup> <https://thedriven.io/2023/01/11/australian-ev-sales-data-for-2022-by-model-with-2023-boom-to-follow/>

<sup>2</sup> <https://www.theguardian.com/environment/2023/mar/05/power-vacuum-how-a-lack-of-charging-stations-is-holding-back-australias-ev-revolution>

## 2. System setting

Consider an EV charging provider overseeing 12 charging stations positioned along the M31 road between Sydney and Melbourne (as indicated in **Table 1**). These charging stations are equipped with varying numbers of chargers, allowing multiple EVs to charge concurrently.

**Table 1: Charging stations between Sydney and Melbourne on M31**

| City id | City/charging station | Distance to last city (km) | Number of chargers installed |
|---------|-----------------------|----------------------------|------------------------------|
| 0       | Sydney                | 0                          | 10                           |
| 1       | Campbelltown          | 57                         | 4                            |
| 2       | Mittagong             | 60                         | 3                            |
| 3       | Goulburn              | 83                         | 4                            |
| 4       | Yass                  | 86                         | 2                            |
| 5       | Gundagai              | 99                         | 3                            |
| 6       | Holbrook              | 115                        | 2                            |
| 7       | Albury                | 62                         | 4                            |
| 8       | Wangaratta            | 74                         | 3                            |
| 9       | Euroa                 | 87                         | 3                            |
| 10      | Wallan                | 106                        | 2                            |
| 11      | Melbourne             | 62                         | 8                            |
|         | <b>Total distance</b> | <b>891 km</b>              |                              |

An EV user seeking to recharge their vehicle must initiate a charging request with the charging supply company. We are developing an application for the company to gather all incoming charging demands (randomly generated for this assignment) within a specified timeframe and allocate charging stations to each EV based on their starting point, destination, battery capacity, and current state of charge (SoC)<sup>3</sup>. Ideally, the allocation should ensure that every EV can travel to its destination with sufficient power while minimising overall waiting time of EVs.

For the sake of programming simplicity, we make the following assumptions:

- *All considered EVs commence their journey from Sydney and exit to a city along the M31 route to Melbourne.* Consequently, the default origin of all EVs is Sydney, except when they pause for charging at a city along the route<sup>4</sup>.
- *Each city under consideration has a single charging station.* This assumption allows us to associate a city's name with its corresponding charging station.
- *A vehicle may need to recharge multiple times to travel to its destination. However, if its remaining battery is sufficient, no charging will be allocated.*
- *Once a vehicle is charged, its battery level will be back to its full capacity.*
- *Each charging takes half an hour, irrespective of the amount of power to charge.*

---

<sup>3</sup> SoC represents the current level of car's battery charge. It determines how long the car can travel without charging.

<sup>4</sup> It is easy to extend your program to apply to traffic flows in both directions.

- *Individual vehicles do not have the autonomy to select their charging stations.*

### 3. Task specification

This assignment is to write a simulation program that can generate randomly a list of EV charging demands and assign EVs to charging stations if charging is needed. The application can be implemented step by step via the following tasks<sup>5</sup>:

**Task 1 (15%):** Create a class, **ChargingStation**, to specify locations (cities and chargers). The class should include at least the following data members:

```
int cityId; //as shown in Table 1
string cityName;
int distanceToLastCity;
int numberOfChargers;
```

The class should include a constructor to initialize the data members with the information in **Table 1** and a function to display the information of a city/charging station. The data have been coded in the file **Constant.h**. Put it as a header file so that your class can use it. Create a class driver (an object of this class in `main()` function) to display the information. *Add any additional data members and member functions for future tasks<sup>6</sup>.*

**Task 2 (15%):** Create a class, **Vehicle**, to specify the information of an EV and its trip. The class should include at least the following data members:

```
int vehicleId; //can be any integer
int currentCityId; //initialised with 0 for Sydney
int destinationId; //any city other than Sydney
int capacityRange; //in kilometers
int remainRange; //in kilometers
```

The class should include a constructor to initialize the data members and a function to display the information of a vehicle. Create a class driver to test this class with any data you specify and print the information accordingly. *Add any additional data members and member functions for future tasks.*

**Task 3 (5%):** Add a function, `distanceToSydney(int cityId)`, to **ChargingStation** class to calculate the distance from Sydney to each city. For instance, `distanceToSydney(2)` should return 117 because the distance from Sydney to Mittagong is 117km.

**Task 4 (10%):** Add a function to your **Vehicle** class to calculate the farthest city a vehicle can travel with its remaining battery from its current location. If resetting current location and remaining battery, the function can also determine the next charging station

---

<sup>5</sup> You do not need to follow the order of the tasks.

<sup>6</sup> For instance, `getNumberOfChargers(cityId)`, `getNameOfCity(cityId)`, `setOrigin(cityId)`, `setRemainRange()`.

after a recharge. In addition, it can be used to check if the current battery level is sufficient to travel to the destination of a vehicle.

**Task 5 (15%):** Create a class called **DemandGenerator** to generate a list of EV charging demands and write the list into a text file. The number of demands can be random between 150 to 200. Each demand is represented in the following form:

[vehicleId, destinationId, capacityRange, remainRange]

For instance, a generated demand [213, 6, 434, 386] means vehicle #213 will travel from Sydney to Holbrook. The battery capacity of this vehicle is 434km. Its remaining battery level can travel up to 386km.

You may generate vehicle ids automatically using continuous integers from a number, say 200<sup>7</sup>. The range of battery capacity can be randomly generated with a value between 350 to 550. The remaining battery travel range can be a random number between 300 to the capacity of the vehicle. All these settings have been specified in the file **Constant.h**. Please keep these settings so that your program is compatible with the demand list we use while demonstration. Download an example of demand list, **chargingDemands.txt**, from vUWS to test your program.

**Task 6 (25%):** Create a class called **ChargingAllocation** to implement the following functionalities:

- Read a demand list from a text file (as generated from **DemandGenerator**) and store them into an array (or vector) of *Vehicle*. The **currentCityId** is set as 0, representing Sydney, initially. You are highly recommended to use vector to store the vehicle objects for flexibility.
- Based on the demand list, determine at which cities a vehicle needs to be recharged (assuming that each will be charged at the furthest city within the remainRange). Some of the vehicles do not need to recharge, some of them need to recharge once and some of them recharge twice. With the current setting, there is no need for recharge three times.
- Calculate the queue length at each city's charging station, i.e., the number of vehicles that will be charged at that charging station.
- Calculate the *average waiting time* for each vehicle at each charging station based on the following formula:

$$Averagewaitingtime = 0.5 \frac{hours * queuelength}{numberofchargers}$$

- Display the above information of each vehicle in the demand list.

**Task 7 (15%):** Improve your charging allocation to reduce overall waiting time. You may find your own strategies to balance charging queues by allowing some vehicle

---

<sup>7</sup> Not recommend using random numbers as vehicle ids.

charging earlier to balance waiting time at each city. The way to calculate *overall waiting time* per vehicle is the following:

$$Overallwaitingtime = \frac{1}{n} \sum_{i=0}^{11} q_i * t_i$$

Where  $n$  is the total number of vehicles queue for charging (including double charging), i.e.,

$$n = \sum_{j=0}^{11} q_j$$

$q_i$  and  $t_i$  are the *queue length* and *average waiting time* at the charging station of each city  $i$  ( $0 \leq i < 11$ )<sup>8</sup>, respectively, as calculated in Task 6.  $\Sigma$  means the sum.

**Note that you may use any approach to reduce overall waiting time, but your approach must be generic so that it can balance waiting time with any given demand list.**

**Task 8 (10% bonus mark):** Convert your C++ code into Java or Python. You must have a pass level C++ code to be eligible for the bonus mark. The maximal marks you can receive is 100%.

### 3. Deliverables

#### 3.1 Declaration

There is no requirement for documentation. However, you are required to include the following declaration as a comment in the file with the main function.

##### **DECLARATION**

*I hold a copy of this assignment that I can produce if the original is lost or damaged.*

*I hereby certify that no part of this assignment has been copied from any other student's work or from any other source except where due acknowledgement is made in the assignment. No part of this assignment has been written/produced for me by another person except where such collaboration has been authorised by the subject lecturer concerned.*

#### 4. Submission

Both the declaration and source code must be submitted via vUWS before the deadline. Your program (.h, .cpp) can be put in separate files (executable file is not required). All these files must be zipped into one file **with your student id as the zipped file name**. Specify in the main source file (.cpp) what compiler/IDE you use make sure your program can be compiled under the compiler/IDE you specified.

**Email submission is not acceptable.**

---

<sup>8</sup> No recharge is needed in Melbourne.

## 5. Demonstration

You are required to demonstrate your program during **your scheduled** practical session in Week 13. **You will receive no marks if you fail the demonstration, especially if you miss the demo time.** Note that it is students' responsibility to get the appropriate compilers or IDEs to run their programs. The feedback to your work will be delivered orally during the demonstration. No further feedback or comments are given afterwards.

The demonstration program must be the same as the one you submit.